Productive Struggle or Silent Stalling? Telemetry Signals of Help-Seeking in LLM-Assisted CS1

Hannah Ostrowski, Mateo Lindqvist, Aisha Bello, and Ravi Chandrasekar — arXiv:2507.01842v1 [cs.HC], 3 Jul 2025

Abstract

Novice programmers who use large language model (LLM) assistants alternate between productive struggle and unproductive stalling, yet instructors rarely see which is which. We instrumented an introductory programming course (N=212) with editor and assistant telemetry and asked teaching assistants to label 1,904 thirty-second windows as productive struggle, stalling, help-seeking, or off-task. A gradient-boosted classifier over keystroke, run, and prompt features reached 0.69 macro-F1 against TA labels. Stalling was best predicted by repeated near-identical prompts and long idle gaps after an error, while productive struggle co-occurred with frequent small edits and test runs. We discuss what instructors can and cannot infer from telemetry, and release our labeling guide.

1 Introduction

LLM assistants are now a default part of many CS1 courses. Their availability changes when and how students ask for help: a student who once raised a hand after twenty minutes can now ask an assistant after twenty seconds. Prior work on productive failure and desirable difficulties suggests some struggle is valuable, but distinguishing valuable struggle from stalling has relied on think-alouds and observation, which do not scale.

We ask: RQ1, which telemetry signals distinguish productive struggle from stalling? RQ2, how reliably can trained TAs label these states from replayed sessions? RQ3, how well does a simple classifier reproduce TA labels?

2 Related Work

Help-seeking in programming has been studied through office-hours logs, forum posts, and intelligent tutoring systems. Keystroke-level data has been used to predict struggle and dropout. Recent work examines how students prompt code assistants and how instructors perceive these tools. We extend this line by labeling behavioral states in sessions that interleave editor activity and assistant prompts.

3 Method

Participants were students in two sections of an introductory Python course at a large public university. A browser IDE recorded edits, runs, test results and assistant prompts. Each session was cut into thirty-second windows; four TAs labeled windows after a two-hour calibration using our labeling guide (Appendix A).

State	Windows	Share
Productive struggle	612	32%
Stalling	441	23%
Help-seeking	538	28%
Off-task	313	16%

Inter-rater agreement on a 10% overlap sample was Cohen's kappa = 0.62.

4 Results

Repeated near-identical prompts (edit distance under 15% between consecutive prompts) were the strongest single predictor of stalling. Long idle gaps following a failed run were the second. Productive struggle windows showed more small edits per minute and more test runs.

The classifier reached 0.69 macro-F1 with session-level cross-validation. Performance dropped to 0.58 when windows from the same student appeared in both train and test folds were removed from the comparison baseline, which we report to make leakage visible.

5 Discussion

Telemetry can flag likely stalling but cannot tell an instructor why a student stalled. We recommend surfacing flags as prompts for a human check-in rather than automatic interventions. Hint timing matters: hints offered during productive struggle were rated as less helpful by students in a follow-up survey.

6 Limitations

One institution, one course, one assistant. TA labels are themselves an imperfect proxy for students' experience.

References

[1] Kapur, M. Productive failure. Cognition and Instruction, 2008.

[2] Bjork, R. A. Memory and metamemory considerations in the training of human beings, 1994.

[3] Price, T. W. et al. Factors influencing students' help-seeking behavior while programming with human and computer tutors. ICER 2017.